

**ĐẠI HỌC QUỐC GIA HÀ NỘI**  
**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



# **BÁO CÁO BÀI TẬP LỚN**

*Học phần: Lập trình nâng cao - UET.CS2043*

*Đề tài: Phát triển hệ thống đấu giá trực tuyến*

*Nhóm sinh viên thực hiện: Nhóm 5*

| STT | Họ và tên          | MSSV     |
|-----|--------------------|----------|
| 1   | Nguyễn Quốc Thái   | 25020389 |
| 2   | Nguyễn Viết Thông  | 25020410 |
| 3   | Đinh Anh Vũ        | 25020431 |
| 4   | Phạm Hữu Chí Thành | 25020396 |

*Lớp: IT1*

## MỤC LỤC

|                                                               |    |
|---------------------------------------------------------------|----|
| <b>Mục lục</b> .....                                          | 2  |
| <b>Lời nói đầu</b> .....                                      | 3  |
| <b>Phần 1: Giới thiệu mục tiêu và phạm vi thực hiện</b> ..... | 4  |
| <b>Phần 2: Kiến trúc tổng thể và mô tả</b> .....              | 7  |
| <b>Phần 3: Đánh giá chức năng theo tiêu chuẩn</b> .....       | 9  |
| <b>Phần 4: Phân chia công việc</b> .....                      | 14 |
| <b>Kết luận</b> .....                                         | 15 |

## **LỜI NÓI ĐẦU**

Đầu giá trực tuyến là một đề tài quen thuộc, tuy nhiên đây vẫn luôn là một bài toán hóc búa đối với sinh viên ngành Công nghệ thông tin về khả năng xử lý thời gian thực và đảm bảo tính hội tụ dữ liệu. Nhận đề tài này, nhóm chúng em đã nỗ lực không chỉ hoàn thành các tính năng cơ bản mà còn đi sâu nghiên cứu các giải pháp nâng cao để mang lại sự hoàn thiện nhất cho hệ thống.

Quá trình thực hiện bài tập lớn là cơ hội quý báu để nhóm vận dụng những kiến thức đã học vào một bài toán thực tế đầy thách thức. Chúng em xin chân thành cảm ơn thầy cô đã tận tình định hướng, giúp nhóm hoàn thiện khung kiến trúc phần mềm một cách khoa học và tối ưu.

Trong quá trình thực hiện đề tài và làm báo cáo, không tránh khỏi những sai sót, chúng em rất mong cô có thể góp ý để nhóm chúng em cải thiện hơn.

Chúng em xin trân trọng cảm ơn!

**Nhóm thực hiện: Nhóm 5**

## PHẦN 1: GIỚI THIỆU MỤC TIÊU VÀ PHẠM VI THỰC HIỆN

### 1. Mục tiêu dự án

- Phát triển hệ thống đấu giá trực tuyến Client-Server thời gian thực, đảm bảo minh bạch trong giao dịch tài chính.
- Tối ưu khả năng chịu tải và tốc độ phản hồi bằng TCP Socket kết hợp đa luồng.
- Đồng bộ dữ liệu tức thời đến tất cả người dùng thông qua Observer Pattern và Server-Push.
- Đảm bảo tính chính xác và an toàn tài chính bằng kỹ thuật kiểm soát đồng thời (ngăn tranh chấp dữ liệu), logic khóa sổ dư tạm thời và biểu đồ động.

### 2. Phạm vi thực hiện

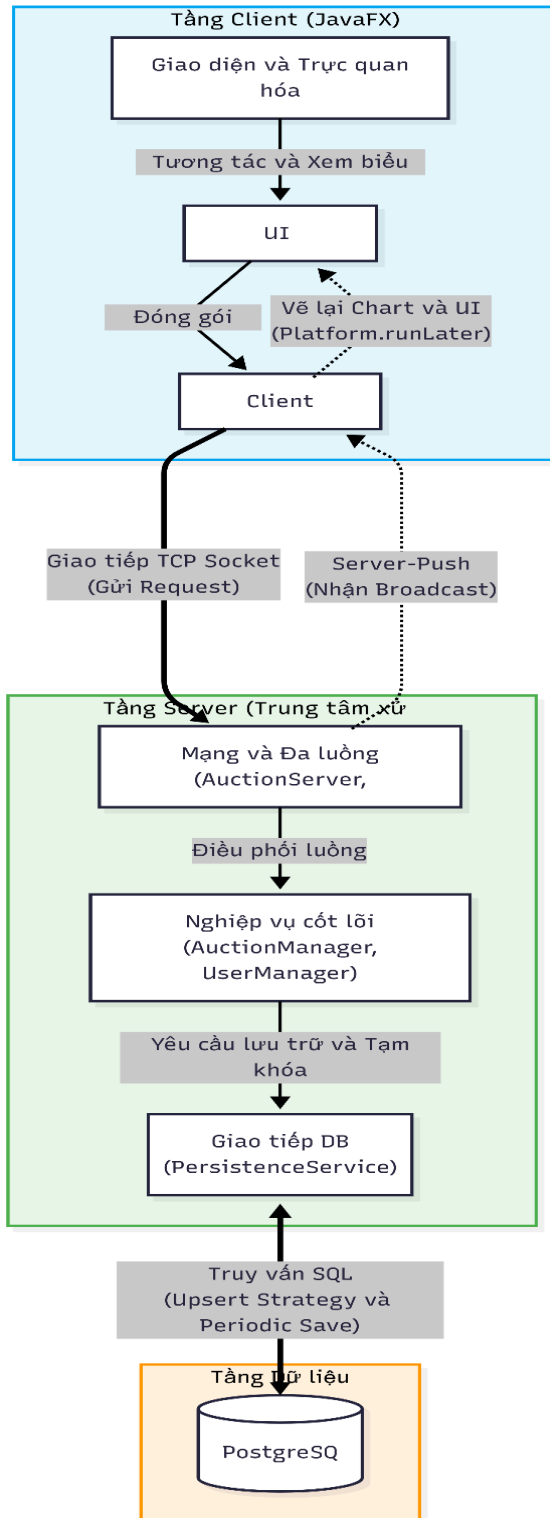
**Về quy trình nghiệp vụ:** Bao quát vòng đời phiên đấu giá qua 4 giai đoạn:

- **Chuẩn bị:** Quản lý xác thực người dùng và tích hợp ví điện tử (nạp tiền, xem số dư).
- **Khởi tạo:** Tạo lập và phân loại danh mục sản phẩm linh hoạt bằng Factory Method.
- **Vận hành:** Hỗ trợ đặt giá thời gian thực và tích hợp cơ chế "Anti-sniping" để tự động gia hạn khi có bid sát giờ.
- **Kết thúc:** Tự động kết toán, thanh toán cho người bán và hoàn sổ dư cho các bên liên quan.

**Về nền tảng kỹ thuật:**

- **Kiến trúc và Giao diện:** Sử dụng Java và JavaFX thiết kế theo kiến trúc phân tầng (tách biệt UI, logic và dữ liệu).
- **Giao tiếp và Lưu trữ:** Dùng TCP Socket thuần cho giao tiếp mạng; quản lý dữ liệu bằng PostgreSQL kèm cơ chế sao lưu dự phòng.
- **Triển khai:** Vận hành ổn định trên mạng nội bộ và sẵn sàng mở rộng trên Internet.

## PHẦN 2: KIẾN TRÚC TỔNG THỂ VÀ MÔ TẢ



### 1. Tổng quan kiến trúc hệ thống

Hệ thống Đấu giá trực tuyến được xây dựng theo mô hình Client–Server kết hợp kiến trúc phân tầng nhằm tách biệt giao diện, xử lý nghiệp vụ và lưu trữ dữ liệu, giúp dễ bảo trì và mở rộng. Hệ thống sử dụng TCP Socket thuần để tối ưu hiệu năng truyền tải, giao tiếp thông qua các đối tượng Request/Response và cơ chế Server-Push nhằm cập nhật dữ liệu thời gian thực.

### 2. Chi tiết các tầng chức năng

#### Tầng Client (Giao diện và Tương tác)

Ứng dụng được phát triển bằng JavaFX với giao diện hiện đại sử dụng FXML và CSS tùy chỉnh. Các Controller quản lý vòng đời giao diện và áp dụng cơ chế cập nhật bất đồng bộ thông qua

**Platform.runLater()** để tránh treo ứng dụng. Thành phần **ClientManager** chịu trách nhiệm quản lý Socket, giao tiếp với Server và duy trì trạng thái đăng nhập

Hình 1: Kiến trúc tổng thể của dự án hệ thống đấu giá trực tuyến

### Tầng Server (Trung tâm xử lý đa nhiệm)

Server hoạt động theo cơ chế đa luồng, mỗi Client được xử lý bởi một **ClientHandler** riêng để đảm bảo khả năng phục vụ đồng thời. Các dịch vụ như **AuctionManager** và **UserManager** quản lý logic nghiệp vụ, xác thực và điều phối dữ liệu trong bộ nhớ RAM. Hệ thống dùng **ConcurrentHashMap** và **synchronized** để kiểm soát đồng thời, ngăn chặn lỗi Race Condition khi nhiều người đặt giá cùng lúc.

### Tầng Dữ liệu (Lưu trữ và đồng bộ dữ liệu)

**PersistenceService** đóng vai trò kết nối giữa RAM và PostgreSQL. Hệ thống áp dụng chiến lược **ON CONFLICT DO UPDATE** để tối ưu lưu trữ dữ liệu và sử dụng cơ chế sao lưu định kỳ nhằm đảm bảo an toàn dữ liệu khi xảy ra sự cố.

## 3. Kịch bản luồng Dữ liệu (Chức năng Đặt giá)

Khi người dùng nhấn “Đặt giá”, **ItemDetailsController** sẽ kiểm tra dữ liệu đầu vào và gửi Request đặt giá đến Server thông qua **ClientManager**.

Tại Server, **ClientHandler** tiếp nhận yêu cầu và gọi logic **AuctionManager** xử lý nghiệp vụ. Phương thức **processBid** trong lớp **Auction** được **synchronized** để kiểm soát đồng thời, đảm bảo kiểm tra số dư, thực hiện cơ chế “Frozen Balance” và cập nhật giá cao nhất một cách an toàn.

Sau khi xử lý thành công, **PersistenceService** ghi dữ liệu giao dịch vào PostgreSQL để đảm bảo tính bền vững của dữ liệu. Server tiếp tục gửi Response xác nhận cho người đặt giá và broadcast thông tin cập nhật đến các Client đang theo dõi phiên đấu giá.

Ở phía Client, **ClientManager** nhận dữ liệu mới, đồng bộ đối tượng **Auction** cục bộ và kích hoạt **notifyObservers()**. Giao diện được cập nhật thông qua **Platform.runLater()**, bao gồm giá hiện tại, lịch sử đấu giá và biểu đồ biến động giá theo thời gian thực.

## PHẦN 3: ĐÁNH GIÁ CHỨC NĂNG THEO TIÊU CHUẨN

### I. CHỨC NĂNG ĐẠT ĐƯỢC

| Chức năng                         | Điểm | Trạng thái | Chi tiết triển khai                                                                                                                        |
|-----------------------------------|------|------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Thiết kế lớp và phân cấp       | 0.5  | Hoàn thành | Hệ thống tổ chức các lớp thực thể từ Entity, Item và NormalUser theo cấu trúc rõ ràng, hỗ trợ mở rộng sản phẩm và người dùng linh hoạt.    |
| 2. Vận dụng OOP                   | 1.0  | Hoàn thành | Áp dụng đầy đủ Encapsulation, Inheritance, Polymorphism và Abstraction trong quản lý dữ liệu, xử lý đấu giá và đồng bộ hệ thống.           |
| 3. Mẫu thiết kế                   | 1.0  | Hoàn thành | Sử dụng Singleton, Factory Method, Observer và Decorator để quản lý đối tượng, cập nhật thời gian thực và mở rộng vai trò người dùng động. |
| 4. Quản lý người dùng và sản phẩm | 1.0  | Hoàn thành | Hỗ trợ đăng ký, đăng nhập, phân quyền linh hoạt, cơ chế Frozen Balance và chống shill bidding.                                             |
| 5. Chức năng đấu giá cốt lõi      | 1.0  | Hoàn thành | AuctionManager quản lý phiên đấu giá, kiểm tra điều kiện bid và lưu lịch sử giao dịch BidTransaction.                                      |
| 6. Xử lý lỗi và ngoại lệ          | 1.0  | Hoàn thành | Triển khai Custom Exceptions và sử dụng ConcurrentHashMap để đảm bảo an toàn dữ liệu đa luồng.                                             |

|                                |     |            |                                                                                                            |
|--------------------------------|-----|------------|------------------------------------------------------------------------------------------------------------|
| 7. Hạ tầng đấu giá đồng thời   | 1.0 | Hoàn thành | Hệ thống hoạt động theo mô hình Client-Server qua Socket TCP kết hợp ThreadPool quản lý kết nối client.    |
| 8. Xử lý đấu giá đồng thời     | 1.0 | Hoàn thành | Phương thức processBid được synchronized nhằm tránh Race Condition và đảm bảo tính toàn vẹn giao dịch.     |
| 9. Cập nhật Real-time          | 0.5 | Hoàn thành | Server broadcast dữ liệu tới Client giúp giao diện cập nhật giá và trạng thái đấu giá theo thời gian thực. |
| 10. Kiến trúc Client-Server    | 0.5 | Hoàn thành | Hệ thống triển khai theo mô hình phân tán với giao tiếp dữ liệu JSON giữa Server và JavaFX Client.         |
| 11. Mô hình MVC                | 0.5 | Hoàn thành | Ứng dụng tuân thủ mô hình MVC giúp tách biệt giao diện FXML, điều khiển và xử lý dữ liệu.                  |
| 12. Maven và Coding Convention | 0.5 | Hoàn thành | Maven quản lý build và thư viện phụ thuộc, mã nguồn tuân thủ camelCase và cấu trúc package khoa học.       |
| 13. Kiểm thử Unit Test         | 0.5 | Hoàn thành | Sử dụng JUnit 5 để kiểm thử logic nghiệp vụ và xác minh quá trình lưu trữ dữ liệu PostgreSQL.              |
| 13. Tự động hóa CI/CD          | 0.5 | Hoàn thành | GitHub Actions tự động build và chạy test nhằm phát hiện sớm lỗi tích hợp hệ thống.                        |



**Chức năng nâng cao**

| Chức năng                       | Điểm | Trạng thái | Chi tiết triển khai                                                                                                                                                |
|---------------------------------|------|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Cơ chế “Anti-sniping”        | 0.5  | Hoàn thành | Hệ thống tự động gia hạn phiên đấu giá thêm 5 phút nếu ghi nhận lượt đặt giá mới vào phút chót nhằm bảo vệ quyền lợi người bán và tạo sự công bằng cho người mua.  |
| 2. Cơ chế trực quan hoá dữ liệu | 0.5  | Hoàn thành | Biểu đồ đường được cập nhật liên tục qua cơ chế Observer để trực quan hóa xu hướng biến động giá thầu và giúp người dùng theo dõi chính xác.                       |
| 3. Cơ chế “Real-time Broadcast” | 0.5  | Hoàn thành | Cơ chế Server-Push qua TCP Socket được áp dụng để chủ động phát sóng mọi thay đổi về thầu giá và trạng thái phiên tới toàn bộ Client đang trực tuyến ngay lập tức. |
| 4. Cơ chế “Frozen Balance”      | 0.5  | Hoàn thành | Hệ thống thực hiện tạm khóa số dư để đảm bảo khả năng thanh toán và tự động hoàn tiền cho người giữ giá cũ ngay khi có người dùng khác trả mức giá cao hơn.        |

**II. HƯỚNG GIẢI QUYẾT VÀ LÝ DO LỰA CHỌN****1. Kiến Trúc Client-Server qua Socket và JSON**

**Hướng giải quyết:** Sử dụng Java Socket TCP để duy trì kết nối thời gian thực giữa Server và nhiều Client, dữ liệu trao đổi dưới dạng JSON bằng thư viện Gson.

**Lý do:**

- Độ trễ thấp, phản hồi bid gần như tức thì.
- Server có thể broadcast dữ liệu mới tới Client theo thời gian thực.
- JSON linh hoạt, dễ mở rộng Request/Response mà không ảnh hưởng kiến trúc mạng.

## **2. Thiết kế OOP và Decorator Pattern**

**Hướng giải quyết:** Xây dựng hệ thống theo OOP với lớp cơ sở Entity và sử dụng Decorator Pattern để quản lý vai trò Bidder, Seller, Admin.

**Lý do:**

- Tránh tạo quá nhiều lớp kế thừa cố định.
- Cho phép gán vai trò động tại runtime.
- Tách biệt trách nhiệm từng vai trò, giúp mã nguồn dễ bảo trì.

## **3. Xử lý đồng thời và Thread Safety**

**Hướng giải quyết:** Kết hợp ConcurrentHashMap, synchronized và ThreadPool

**Lý do:**

- Ngăn Race Condition khi nhiều người cùng đặt giá.
- Đảm bảo logic Frozen Balance và hoàn tiền được xử lý nguyên tử.
- ThreadPool giúp tối ưu tài nguyên và tránh quá tải Server.

## **4. Cơ chế Anti-Sniping**

**Hướng giải quyết:** Tự động gia hạn 5 phút nếu có bid trong 5 phút cuối.

**Lý do:**

- Ngăn hành vi “ép giá” ở giây cuối.
- Tạo cạnh tranh công bằng và tối ưu lợi nhuận cho người bán.

## **5. Observer Pattern cho Real-time UI**

**Hướng giải quyết:** Auction duy trì danh sách observers và gọi notifyObservers() khi giá hoặc trạng thái thay đổi.

**Lý do:**

- Tách biệt Model với View/Controller theo nguyên lý OOP.
- Kết hợp Platform.runLater() để cập nhật JavaFX an toàn trong môi trường đa luồng.

## **6. Lưu trữ dữ liệu bền vững**

**Hướng giải quyết:** Sử dụng PostgreSQL trên Docker kết hợp PersistenceService để lưu định kỳ và lưu khi shutdown hệ thống.

**Lý do:**

- Đảm bảo dữ liệu không mất khi Server gặp sự cố hoặc khởi động lại.
- Docker giúp đồng bộ môi trường phát triển và triển khai ổn định trên mọi máy.

## PHẦN 4: PHÂN CHIA CÔNG VIỆC

| Nhiệm vụ thực hiện      | Thành viên                   | Chi tiết kỹ thuật cốt lõi                                                                                                                                 |
|-------------------------|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Tầng Dữ liệu         | Quốc Thái, Anh Vũ            | Định nghĩa thực thể; áp dụng tính kế thừa/đa hình cho sản phẩm; thiết lập logic trạng thái phiên đấu giá trong lớp Auction.                               |
| 2. Tầng Nghiệp vụ       | Quốc Thái, Viết Thông        | Triển khai Singleton Managers; xử lý logic đặt giá processBid (frozen balance); luồng monitorState giám sát trạng thái thời gian thực.                    |
| 3. Tầng Database        | Viết Thông                   | Giao tiếp PostgreSQL qua JDBC; sử dụng <b>Reflection</b> để hydrate dữ liệu; thiết lập <b>ScheduledExecutorService</b> lưu trữ định kỳ và dự phòng sự cố. |
| 4. Tầng Mạng            | Quốc Thái, Anh Vũ, Chí Thành | Xây dựng hạ tầng TCP Socket đa luồng; phân giải JSON; triển khai cơ chế <b>Server-Push (Broadcast)</b> cập nhật dữ liệu tới toàn bộ Client.               |
| 5. Tầng UI/UX           | Chí Thành                    | Thiết kế giao diện FXML; áp dụng CSS; tối ưu trải nghiệm người dùng trực quan.                                                                            |
| 6. Điều khiển và Client | Quốc Thái, Anh Vũ, Chí Thành | Duy trì kết nối qua ClientManager; xử lý sự kiện tại Controller; tích hợp biểu đồ động cập nhật biến động giá thời gian thực.                             |
| 7. Kiểm thử và DevOps   | Viết Thông, Anh Vũ           | Viết Unit Test với JUnit; cấu hình Maven; thiết lập môi trường DB ảo với <b>Docker Compose</b> và tự động hóa CI/CD qua GitHub Actions.                   |

## **KẾT LUẬN**

Qua quá trình thực hiện đề tài “Auction System”, nhóm chúng em đã hoàn thành được các yêu cầu nghiệp vụ và kỹ thuật đặt ra, đồng thời có cơ hội vận dụng kiến thức đã học vào một hệ thống mang tính thực tiễn cao.

Dự án giúp chúng em tích lũy thêm nhiều kinh nghiệm trong việc xây dựng hệ thống Client–Server bằng Socket thuần, xử lý đa luồng, đồng bộ dữ liệu thời gian thực và áp dụng tư duy hướng đối tượng trong thiết kế phần mềm. Qua đó, nhóm hiểu rõ hơn tầm quan trọng của tính chính xác, ổn định và khả năng phối hợp khi phát triển các hệ thống giao dịch.

Mặc dù đã cố gắng hoàn thiện sản phẩm và báo cáo một cách tốt nhất, bài làm chắc chắn vẫn còn những thiếu sót nhất định. Nhóm chúng em rất mong nhận được những góp ý để có thể tiếp tục hoàn thiện và phát triển hơn trong tương lai.

Cuối cùng, nhóm chúng em xin chân thành cảm ơn thầy cô đã tận tình hướng dẫn và hỗ trợ trong suốt quá trình thực hiện đề tài. Học phần Lập trình nâng cao đã mang lại cho chúng em nhiều kiến thức và trải nghiệm thực tế quý báu cho quá trình học tập cũng như định hướng nghề nghiệp sau này.

**Nhóm thực hiện: Nhóm 5**